



FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

TEST 1 MEMO

DATE: 24 MARCH 2026

TIME: 1 ½ Hour

EXAMINER: BOAMAH-ABU, CHARLES (PhD)

TOTAL MARKS: 50

NO. OF PAGES: 6 (Including cover page)

INSTRUCTIONS/REQUIREMENTS

1. *This examination is closed book*
2. *Answer all questions*
3. *Where stated "Briefly answer", please give a short answer*
4. *Where stated "Write the missing parts" of code, please write only the missing parts*
5. *Ensure your handwriting is readable*

Question 1

10 marks

In each of the following, indicate whether the statement is true or false. Write **T** (for **True**) or **F** (for **False**).

1. Inheritance reduces duplication of code.
2. Code reuse increases software development cost.
3. Any class can be declared abstract.
4. An abstract class can inherit another abstract class.
5. An abstract class can contain instance variables.
6. An interface can contain constructors.
7. An interface can contain ONLY abstract methods.
8. An interface is mainly for code reuse not instantiation.
9. A class can implement two or more interfaces.
10. Design pattern are programs to solve common software development problem.

1. T 2. F 3. T 4. T 5. T 6. F 7. F 8. T 9. T 10. F

Question 2

12 marks

Briefly answer the following questions.

1. What is the main benefit of Integrative programming? (2)

Answer: any correct answer e.g., allows different applications of different platforms to work together; allows interoperability) ✓✓

2. You are to write a program for the classes below. What is wrong with the design? (2)

PartTime	FullTime
-name: String -staffId: String -numHours: int +calcMonthlyPay(): double	-name: String -staffId: String -annualSal: double +calcMonthlyPay(): double

Answer: some members are duplicated ✓✓

3. What is wrong with the code segment below? (2)

```
...
public abstract class Example {
    ...
    public abstract int a (int x, int y);
    public abstract double a (int x, int y);
}
```

Answer: Violates method overload (same method name but not different parameter list) ✓✓

4. **I** is an interface; **A** is an abstract class and **C** is a concrete class. **C** inherits **A** and implements **I**. In each of the following, indicate whether the statement is **valid** or **invalid**. Explain your answer. (6)

- a. `I i1=new I();`
- b. `A a1=new A();`
- c. `A a2=new C();`

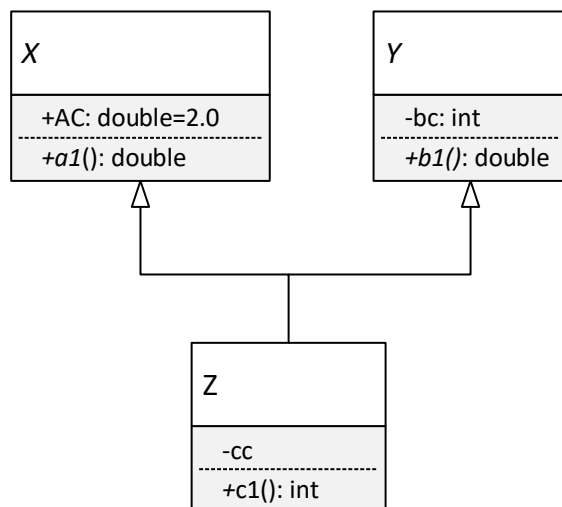
Answer

- a. `I i1=new I();` Invalid, cannot instantiate an interface✓✓
- b. `A a1=new A();` Invalid; cannot instantiate an abstract class✓✓
- c. `A a2=new C();` Valid; can instantiate a concrete class and assign to a variable of a superclass✓✓

Question 3

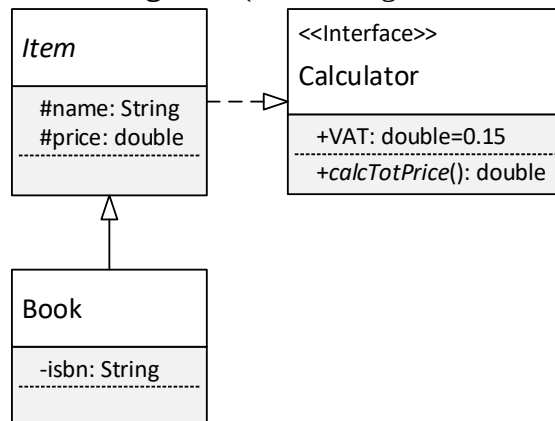
13 marks

3.1. In the diagram below, the concrete class **Z** inherits two abstract classes: **X** and **Y**. Which class can be converted to an interface to resolve multiple inheritance? Explain your answer. (3)



Answer: X can be converted to an interface (Y cannot be converted because it contains an instance variable)✓✓✓

3.2. Below is part of a diagram for a program that calculates the total price of items (e.g., book, phone, etc.) in a shop. The method **calcTotPrice** returns the total price as **price + VAT chargeable** (VAT chargeable is **15% of the price** on an item).



3.2.1. Complete the code for **Calculator**. (5)

```

_____ {
    _____
    _____
}
  
```

Answer: note the long form of variable and method declarations are also correct

```

public interface Calculator ✓{
    double VAT=0.15; ✓✓
    double calcTotPrice(); ✓✓
}
  
```

3.2.2. Write the class header of **Item**. (2)

Answer
 public abstract class Item implements Calculator ✓✓

3.2.3. Write the method **calcTotPrice()** in the class **Item**. (3)

Answer
 public double calcTotPrice() ✓✓{
 return price +VAT*price; ✓
 }

Question 4

15 marks

4.1. Map the following design patterns to their application in software development (write ONLY the **number** of the pattern **and** corresponding **letter** or **letters**). (5)

Answer: C and D; 2. A 3. B 4.

	Design pattern		Application
1	Singleton	A	To create instances of a subclass as instances of superclass
2	Factory	B	To bridge different patterns to make them work together
3	Adapter	C	To manage shared resource
4	Template	D	To create ONLY one instance of an object
		E	To define how subclasses implement the skeleton of an algorithm in a superclass

4.2. Below is the structure of a Singleton class called **Login**, based on **lazy initialization**.

```

G Login
  S log : Login
  S user : String
  S pswd : String
  C Login(String, String)
  S create(String, String) : Login
  S toString() : String

```

log is a static variable, and **user** and **pswd** are instance variables. The parameterized constructor initializes **user** and **pswd**. The method **create** is static; takes **user** and **pswd** as parameters to create and return an instance of **Login**

4.2.1. Write a statement to declare the static variable **log**. (2)

private static Login log; ✓✓

4.2.2. Complete the code (below) for the method **create**. (6)

```

_____create(_____){
    _____
    _____
    _____
}

```

Answer:

```

public static Login create(String user, String pswd) {
    if(log==null){
        log=new Login(user, pswd);
    }
    return log;
}

```

- 4.2.3. The **main** method in the class **LoginTester** (below) calls an appropriate method in **Login** with the values "linono1" and "@212121", assigns an instance of **Login** in the variable **l** and displays **l**. Write the missing code to call the appropriate method. (2)

```
public class LoginTester {  
  
    public static void main(String[] args) {  
        Login l= _____  
        System.out.println(l);  
    }  
  
}
```

```
package login;  
  
public class LoginTester {  
  
    public static void main(String[] args) {  
        Login l= Login.create("linono1", "@212121");✓✓  
        System.out.println(l);  
    }  
  
}
```